14. JSON, Encoding, and Time

14. JSON, Encoding, and Time

Phase: Phase 2 — Core language depth
Estimated time: 1 day
Slug: `14-json-encoding-time`

Learning objectives

- Marshal and unmarshal JSON with struct tags
- Handle time.Time and timezones carefully
- Validate unknown fields when needed
- Avoid floating point for money — use integers

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

JSON and time are the lingua franca of services. Struct tags and time.Time correctness prevent subtle production outages.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- encoding/json uses struct tags (`json:"name"`).
- Unknown fields are ignored by default; unknown types error.
- time.Time marshals as RFC3339 by default.
- Always store and compare times in UTC unless UI demands local.
- Use json.NewDecoder for streams; Marshal for small values.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-json, && cd /tmp/gw-json,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "encoding/json"
    "fmt"
)

type User struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}

func main() {
    b, _ := json.Marshal(User{Name: "Ada", Age: 36})
    fmt.Println(string(b))
    var u User
    _ = json.Unmarshal(b, &u)
    fmt.Println(u)
}
```

Run it:

```
go run .
```

Expected output:

```
{"name":"Ada","age":36}
{Ada 36}
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Round-trip a struct with tags.
2. Omit empty fields with `omitempty`.
3. Parse a timestamp with time.Parse(time.RFC3339, …).

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

JSON

```
type Event struct {
    ID   string    `json:"id"`
    At   time.Time `json:"at"`
    Name string    `json:"name,omitempty"`
}

b, err := json.Marshal(ev)
err = json.Unmarshal(b, &ev)
```

Use `Decoder` for streams; `Unmarshal` for whole buffers.

Time

```
now := time.Now().UTC()
t, err := time.Parse(time.RFC3339, s)
```

Store UTC; convert for display. Durations use `time.Duration` (nanoseconds under the hood).

Encoding discipline

Be explicit about omitempty, pointers for null vs empty, and custom MarshalJSON only when needed.

Failure modes to watch while coding

- Parsing local times without timezone
- Using float64 for currency in JSON
- Silent ignore of unknown fields in security-sensitive configs

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- dict/json vs struct tags — unknown fields won't appear as attributes.
- datetime timezone naivety is worse in Python; still be careful in Go.
- snake_case JSON ↔ CamelCase Go fields via tags.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Validate after unmarshal — zero values can mean missing JSON fields.

Practice checklist

Before the exercise, confirm you can:

1. Explain "JSON, Encoding, and Time" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Parsing local times without timezone
- Using float64 for currency in JSON
- Silent ignore of unknown fields in security-sensitive configs

Exercises

Work in `exercises/14-json-encoding-time/`. Read that folder's `README.md` before you code.

What you will build

Implement JSON round-trip helpers to satisfy the exercise tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/14-json-encoding-time/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does `omitempty` do?
2. Why prefer UTC storage?
3. Decoder vs Unmarshal?

Further reading

- https://pkg.go.dev/encoding/json
- https://pkg.go.dev/time